PROJECT GUIDE · PRESENTATION & VIVA PREPARATION

TESSERA

A Geo-Agentic Real-Time Spatial Stream Analytics pipeline built on Apache Kafka — explained from zero, for someone who has never seen the project.

HOW TO USE THIS GUIDE

Read chapters 1–3 first: they give you the story you tell in the first two minutes of any presentation. Chapters 4–9 explain each technology and each piece of code. Chapter 10 decodes every command output you will show live. Chapter 12 is a ready-made demo script, and chapter 13 is a bank of viva questions with answers.

Boxes marked **SAY THIS** are sentences you can use almost word for word in front of examiners.

1 The problem, in plain English
2 The big idea: two speeds
3 The whole pipeline in one picture
4 Kafka, explained through this project
5 Stream processing: windows & time
6 The statistics: finding hotspots
7 The agent: reasoning without slowing anything
8 Storage and fault tolerance
9 Docker: how it all runs on Windows
10 Every command and what its output means
11 Dashboard tour, panel by panel
12 A 7-minute live demo script
13 Viva questions and answers
14 Code map: what every file does
15 Requirements checklist
16 Honest limitations & future work
17 Glossary

Based on the TESSERA methodology brief (“A Geo-Agentic AI Framework for Real-Time Spatial Stream Analytics”) and the working implementation in the TESSERA project folder.

1 · The problem, in plain English

Imagine a city with thousands of taxis. Every couple of seconds, every taxi sends a tiny message: *"I am taxi-0042, I am at latitude 39.91, longitude 116.40, moving at 34 km/h."* That never-ending river of messages is called a **spatial stream** — "spatial" because every message has a location, "stream" because it never stops.

Somewhere in that river, something important happens: an accident blocks a road and forty taxis suddenly crawl at 4 km/h around one junction. The city wants to know **now**, not in ten minutes. A traffic alert delivered ninety seconds late is not a slightly worse alert — it is useless.

Two kinds of existing tools, and why neither is enough

Kind of tool	What it is good at	What it cannot do
Spatial stream engines e.g. GeoFlink on Apache Flink	Extremely fast. Millions of GPS points per second, with correct handling of time and geometry.	**"Intent-frozen."** A human must write every query in advance ("alert me when a car enters this circle"). If something new happens, the system cannot decide by itself to watch for it.
AI agents for maps (agentic GIS) LLM-based assistants	Understand intent ("find where traffic is jammed and warn drivers"), plan, and write code.	**Slow and one-shot.** One LLM call takes 0.5–5 seconds. They work on a fixed dataset, answer once, and stop.
LLM inside the stream call the model per message	Semantic understanding of every message.	**Collapses on arithmetic.** At 50,000 messages/s and 1 s per call you would need ~50,000 model calls running at the same time. Cost grows with data volume, which you do not control. LLMs also make spatial mistakes (swap latitude and longitude, measure distance in degrees).

> **THE EXACT PROBLEM STATEMENT**
>
> Build a system that analyses an unbounded stream of georeferenced events **in real time**, where an AI agent can **change what the system is looking for** as the situation changes — **without** the agent ever slowing down the data path, **without** the cost of the agent growing with the data volume, and **without** letting the agent's geographic mistakes reach production.

What our implementation demonstrates concretely

- A simulated fleet of **400 taxis in Beijing** sends about **200 GPS events per second** into Apache Kafka.
- Every 2.5 minutes a **traffic incident** is secretly injected somewhere in the city (taxis inside a ~700 m circle slow to 1–7 km/h). The system is not told where.
- The pipeline must **find** the incident from the data alone, have an AI agent **decide** what it is, **change its own monitoring** (add a geofence around it), and raise **live alerts** when taxis drive in — all while the stream keeps flowing at full speed.
- Everything is stored in a time-series database and shown on a live dashboard.

> **SAY THIS**
>
> "Fast streaming systems are blind — they only answer questions a human wrote in advance. AI agents are smart but far too slow to sit inside a stream. TESSERA keeps them separate: the stream processing stays fast and deterministic, and the agent only changes the *program* that processes the stream, never the individual messages."

2 · The big idea: two speeds

The paper calls it the **timescale inversion**. Instead of asking "how do we make the AI fast enough to keep up with the data?", TESSERA says: the AI does not need to keep up, because the AI is not processing the data. It is processing the *program* that processes the data.

An analogy you can use

Think of a busy restaurant kitchen. The **line cooks** (fast plane) cook hundreds of dishes per hour by following recipe cards exactly. They never stop to think. The **head chef** (slow plane, the AI agent) does not cook individual dishes. Every so often she notices something — "we are out of tomatoes", "table 6 has a nut allergy" — and **changes a recipe card**. Before the new card goes to the line, a **food-safety inspector** (the verifier) checks it for mistakes, and the cooks try it on a few test plates first (shadow run). The kitchen never slows down because the chef is thinking.

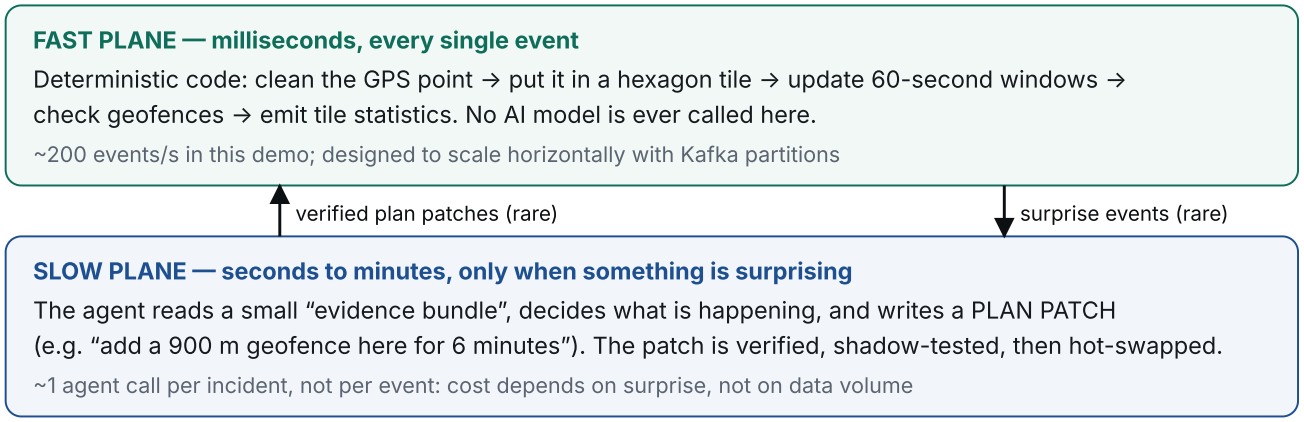


Figure 1 — The two planes only talk through Kafka topics, and only rarely.

Three consequences (these are the paper's main claims)

1. **The agent's latency never enters the data path.** If the AI takes 2 seconds or 30 seconds to think, the GPS events are processed at exactly the same speed. You can prove this live with the "Inject 30 s agent latency" button (hypothesis H1, property P1).
2. **AI cost depends on how surprising the world is, not on how much data arrives.** A statistical gate decides when to wake the agent (property P2). Doubling the number of taxis does not double the number of AI calls.
3. **Because the agent outputs a program, not an answer, the program can be checked before it runs.** "Join stream A within 400 m" can be type-checked for unit errors, swapped coordinates, etc. A claim like "these cars will collide" cannot. This turns an impossible truth-checking problem into an easy type-checking problem (property P5).

Why it is called TESSERA

A *tessera* is a mosaic tile. The system divides the map into hexagonal tiles (using Uber's **H3** grid), and the tile is used for *everything*: which Kafka partition an event goes to, which events need a geometry check, which area is statistically tested, where memory facts are stored, and how far a cached AI decision can be reused. One spatial primitive, five jobs.

> **SAY THIS**
>
> "The model is never an operator in the stream. It is a compiler, a gatekeeper, a curator and a distiller. Its only output type is a verified plan patch."

3 · The whole pipeline in one picture

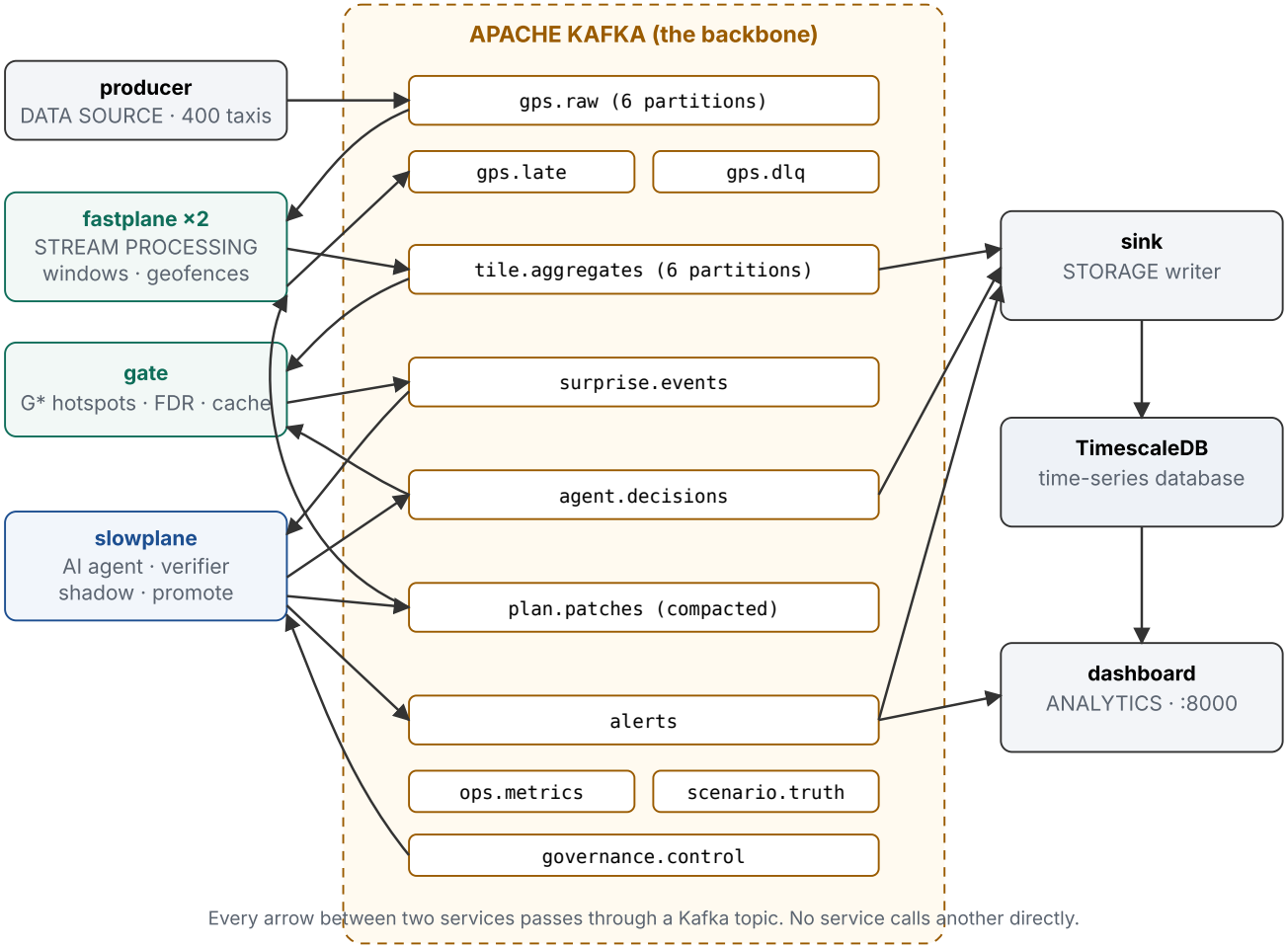


Figure 2 — Data Source → Kafka → Stream Processing → Storage → Analytics/Dashboard.

Follow one incident through the system (the story to tell)

1. **Data source.** The `producer` sends 400 taxi positions every 2 s to topic `gps.raw`. At t≈45 s it secretly starts an incident: taxis inside a ~700 m circle slow to a crawl. It writes the true location to `scenario.truth` only for later scoring.
2. **Stream processing.** Two `fastplane` containers share the 6 partitions of `gps.raw`. Each event is cleaned, assigned to a hexagon tile, and added to 60-second sliding windows. Every 10 s, closed windows produce one summary per tile (count, vehicles, mean speed, share of slow cars) into `tile.aggregates`.
3. **Surprise gate.** The `gate` looks at all tiles of a window at once and runs a hotspot test (Getis-Ord G*) with false-discovery control. The incident tiles score z≈15 — extremely unlikely by chance. It writes an **evidence bundle** to `surprise.events` and a `HOTSPOT_DETECTED` alert.
4. **Slow plane.** The agent reads the bundle and its memory of recent facts, decides “congestion_incident, risk 1.00”, and proposes a **plan patch**: add a geofence of ~800 m around it for 6 minutes.
5. **Safety.** The **verifier** checks the patch (coordinates, units, CRS, window rules…). The **governor** decides if a human must approve. The patch runs in **shadow** for 20 s on the live stream, then is **promoted**: `plan.patches` now holds plan v2.
6. **Hot-swap.** Both fastplane containers load plan v2 at the next window boundary. From now on, every taxi entering the circle produces a `GEOFENCE_ENTER` alert in microseconds, with no AI call.
7. **Storage & dashboard.** The `sink` writes aggregates, alerts and decisions to **TimescaleDB**. The `dashboard` reads Kafka live and the database for history, and draws the map, alerts and charts.

4 · Kafka, explained through this project

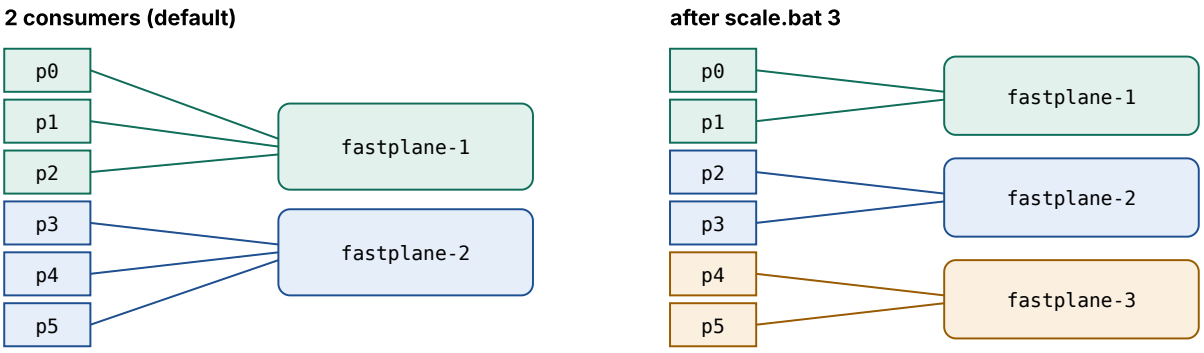
Apache Kafka is a **distributed commit log**: a place where programs append messages and other programs read them, in order, at their own pace. Think of it as a set of append-only notebooks that never lose a page.

The vocabulary, with our examples

Concept	What it means	In TESSERA
Broker	The Kafka server that stores the messages.	One broker, container `kafka` (Apache Kafka 3.8, KRaft mode — no ZooKeeper).
Topic	A named stream of messages, like a table name.	11 topics, e.g. `gps.raw`, `alerts`.
Partition	A topic is split into partitions. Each partition is an ordered log. Partitions are the unit of parallelism: one partition can be read by only one consumer of a group at a time.	`gps.raw` and `tile.aggregates` have **6 partitions** each.
Offset	The position (0, 1, 2, …) of a message inside its partition. A consumer remembers "I have read up to offset 16,793".	Shown in `status.bat` as CURRENT-OFFSET.
Key	Each message can have a key. Kafka hashes the key to choose the partition, so the same key always goes to the same partition.	Key = the **H3 cell (resolution 6, ~3 km)** of the GPS point. All events of one neighbourhood land in the same partition.
Producer	A program that writes messages.	`producer`, `fastplane`, `gate`, `slowplane`, dashboard buttons.
Consumer	A program that reads messages.	`fastplane`, `gate`, `slowplane`, `sink`, `dashboard`.
Consumer group	Several consumers with the same `group.id` **share** the partitions of a topic: each partition goes to exactly one member. Different groups each get **all** messages independently.	`tessera-fastplane`, `tessera-gate`, `tessera-slowplane`, `tessera-sink`.
Rebalance	When a consumer joins or leaves a group, Kafka redistributes the partitions.	Seen with `scale.bat` and the fault-tolerance demo.
Lag	How many messages a consumer is behind (end offset − current offset).	Near 0 means it keeps up.
Retention	How long Kafka keeps messages.	`gps.raw`: 6 hours — this is what makes replay after a crash possible.
Compaction	Keep only the latest message per key instead of deleting by time.	`plan.patches` is compacted: the latest "active" plan is always available to a new instance.

Why partitioning by map area matters (spatial locality)

Window state is per tile. If events of the same tile went to two different consumers, each would see half the cars and compute wrong averages. Because the key is the coarse H3 cell, **every event of a tile always reaches the same consumer**, so per-tile state never has to be shared across machines. This is the same trick GeoFlink uses with its grid index: *partition locality = spatial locality*. You can see it on the map with the **Kafka partition** button: neighbouring hexagons share a colour.



Partitions of gps.raw inside consumer group tessera-fastplane. Kafka moves partitions automatically when a member joins or leaves.

Figure 3 — Consumer group rebalancing. Max useful consumers = number of partitions (6).

The 11 topics and why each exists

Topic	Parts	Written by → read by	Purpose
`gps.raw`	6	producer → fastplane	Raw GPS events. Key = coarse H3 cell. The main input.
`gps.late`	3	fastplane → (Kafka UI)	"Side output" for events that arrived too late for any window. Kept, not silently dropped.
`gps.dlq`	1	fastplane → (Kafka UI)	Dead-letter queue: malformed records (lat=999, missing id, unknown CRS, negative speed) with the reason.
`tile.aggregates`	6	fastplane → gate, sink, dashboard	One summary per tile per closed window.
`surprise.events`	3	gate → slowplane, sink	Evidence bundles for statistically surprising hotspots. The only way the agent is woken up.
`agent.decisions`	3	slowplane, gate → gate, sink, dashboard	Agent judgements (and cache reuses). Feed the cache and the variogram.
`plan.patches`	1, compacted	slowplane → fastplane, dashboard	Plan versions: *shadow*, *active*, *rejected*. Broadcast to every fastplane.
`alerts`	3	all planes → sink, dashboard	Everything the user sees in "Live alerts".
`ops.metrics`	3	every service → sink, dashboard, slowplane	Telemetry: events/s, latency, lag, counters.
`scenario.truth`	1	producer → sink, dashboard	Where incidents really were — used only to measure detection time.
`governance.control`	1	dashboard → slowplane	Human commands: approve/reject a plan, demo controls.

Broadcast vs. load-sharing (a subtle point examiners like)

For `gps.raw` we want **load-sharing**: each event processed once, so all fastplanes use the **same** group. For `plan.patches` we want **broadcast**: every fastplane must receive every plan. So each instance reads plans with its own **unique** group id (`tessera-fastplane-plans-<instance>`). Same Kafka, two patterns, chosen by the group id.

Reliability settings we use (and why)

- **Idempotent producer,** `acks=all` — if a send is retried after a network glitch, Kafka de-duplicates it; messages in a partition are never duplicated or reordered by retries.
- **Manual offset commit** (`enable.auto.commit=false`) — a consumer only says "I am done up to offset X" *after* its results are safely stored. If it crashes before that, the messages are read again.
- **Auto topic creation disabled** — topics are created explicitly by `init` with the right partition counts. Messages are sent in small lz4-compressed batches (20 ms linger).

5 · Stream processing: windows & time

Step 0 — cleaning ("L0 normalise")

Before anything else, every event is validated in `geo.normalise()`. Real fleets send garbage: impossible coordinates, unknown coordinate systems, negative speeds. 10% of our taxis also report coordinates as `lon,lat` instead of `lat,lon` — but they *declare* it, so we swap them correctly instead of guessing. Bad records go to `gps.dlq` with the reason; the pipeline never crashes on them.

Step 1 — tiling with H3 hexagons

H3 (from Uber) divides the Earth into hexagons at 16 resolutions. We use **resolution 8 (edge ≈ 0.5 km)** for analysis tiles, and its parent at **resolution 6 (≈ 3 km)** as the Kafka key. Hexagons are used because every neighbour is at the same distance (squares have diagonal neighbours that are further away), which makes "the ring of tiles around this tile" well defined.

Step 2 — event time, not arrival time

Each GPS event carries the time it was *measured* (event time). It may *arrive* late because of networks. We group by event time, so results are correct even when messages arrive out of order. Our producer deliberately delays 5% of events by 0.5–3.5 s and 0.4% by 75–95 s to prove this works.

Step 3 — sliding windows

A **window** is a time bucket. Ours are **60 seconds long and slide every 10 seconds**, so each event belongs to 6 overlapping windows, and every 10 s a fresh "last minute" summary is produced per tile.

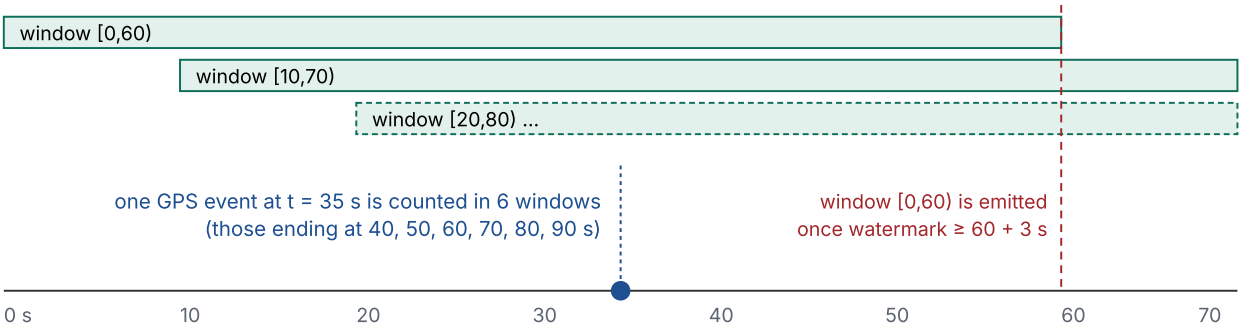


Figure 4 — 60 s windows sliding every 10 s. Each closed window emits one record per tile to tile.aggregates.

Step 4 — watermarks and allowed lateness: "when is a window finished?"

Because events can arrive late, we cannot close the [0,60) window at exactly 60 s. A **watermark** is the system's statement "I believe I have seen everything up to time W". We compute it as *largest event time seen − 4 s* (bounded out-of-orderness). A window is emitted when `window_end + 3 s allowed lateness ≤ watermark`. Events that arrive after their windows were emitted go to `gps.late` (side output) — that is the "Late" number on the dashboard.

Idle partitions: if a partition receives nothing for 15 s, its watermark is advanced with the clock, otherwise one quiet area would block results forever.

Step 5 — aggregation (the summary per tile per window)

Field	How it is computed	Why
`n_obs`	count of events	traffic volume
`vehicles`	number of distinct taxi ids (a set)	one taxi reporting 30 times ≠ 30 taxis
`mean_speed`, `std_speed`	from running sums Σv and Σv² (no need to keep every value)	constant memory per tile — the standard streaming trick
`slow_frac`	share of events with speed < 10 km/h	the congestion signal

Step 6 — continuous geofence operators ("L2") with filter-and-refine

When a plan contains a geofence (circle), checking every event against every circle with exact geometry would be slow. So we first **filter**: precompute the set of hexagons that could possibly touch the circle (a ring of k tiles). An event outside those tiles is skipped without any maths. Only events inside the candidate tiles are **refined** with the exact geodesic (haversine) distance. A vehicle alerts at most once per 120 s per geofence (cooldown).

> **A FINDING WORTH MENTIONING**
>
> The paper's ring formula `k = ⌈r / 2c⌉ + 1` assumes perfect hexagons. Real H3 cells are distorted, and our unit test found cases where it missed cells (a false negative = a missed alert). We replaced it with a bound computed from the *measured* cell size and neighbour spacing, and a test now checks 24,000 random points for zero misses.

6 · The statistics: finding hotspots

The gate answers one question every 10 seconds: *"Is any part of the city behaving surprisingly right now?"* If not, the agent sleeps. This is what keeps AI cost independent of data volume.

The monitored value

For each tile: **x = n_obs × slow_frac** = number of slow reports in the last minute. A busy road with normal speed scores low; a jam scores high.

Getis-Ord G^*_i — "is this tile *and its neighbours* unusually high?"

One slow taxi at a red light should not trigger anything. A real jam makes a **cluster** of neighbouring tiles high at the same time. G^*_i adds up x over tile *i* and its 6 neighbours (k-ring 1), compares the sum with what you would expect if values were spread randomly across the city, and divides by the standard deviation. The result is a **z-score**: z = 2 is mildly unusual, z = 15 (what our incidents produce) is astronomically unlikely by chance. A slowly adapting average (EWMA) of the city baseline is blended in, so the gate adapts to rush hour.

From z to p-value, and the multiple-testing trap

We convert each z into a **p-value** (probability of seeing this by pure chance). With ~400 tiles tested every 10 s, testing each at p < 0.05 would produce ~20 false alarms *every window*. That would wake the agent constantly.

Benjamini–Hochberg (BH) false-discovery-rate control

BH controls the **expected share of false alarms among the alarms raised** (we use q = 0.01, i.e. at most ~1%). Recipe: sort the m p-values from smallest; find the largest rank r with `p(r) ≤ r × q / m`; flag the r smallest.

```
# tiny example, q = 0.05, m = 5 tiles
sorted p:   0.001   0.008   0.039   0.041   0.90
threshold:  0.010   0.020   0.030   0.040   0.050     (rank × 0.05 / 5)
p ≤ thr?    yes     yes     no      no      no        → flag the first 2 tiles
```

This is exactly what the unit test `test_benjamini_hochberg_simple` checks. In the dashboard alert "*G* hotspot: 3 tile(s), z=14.8, p=1.4e-49 (BH q=0.01)*" you see all of it in one line.

From flagged tiles to one evidence bundle

- Flagged tiles with positive z and at least 4 slow reports are grouped into **connected clusters** (tiles touching each other) — one cluster = one hotspot.
- For each cluster the gate first asks the **cache** (chapter 7): "has the agent judged something very similar nearby, recently?" If yes → reuse, no AI call (`CACHED_DECISION`).
- Otherwise a **token bucket** (6 calls/minute, burst 3) limits the call rate. If no token is available the bundle waits in a priority queue (strongest z first) and expires after 60 s.
- Tiles already being analysed are not re-sent for 45 s.

> **WHY THIS MATTERS (PROPERTY P2)**
>
> Expected agent calls per window ≤ $q \cdot m_0 + m_1$: a small fraction of the quiet tiles plus the truly surprising ones.
>
> More taxis make the statistics *sharper*, not the AI bill larger. On the dashboard, compare "11,543 tile tests" with "8 agent invocations".

7 · The agent: reasoning without slowing anything

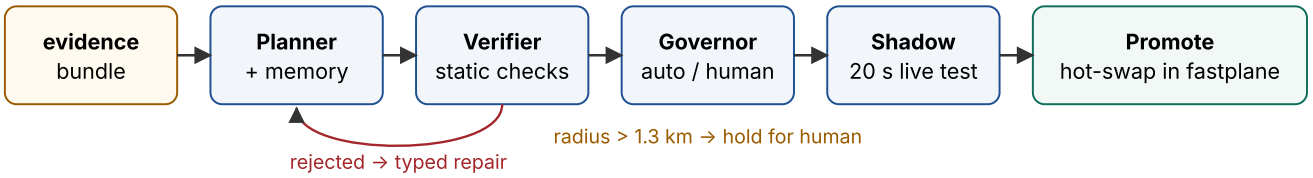


Figure 5 — The life of a plan patch in the slow plane (services/slow_plane.py).

7.1 The plan: GS-IR (Geo-Stream Intermediate Representation)

The running query is a small JSON document with a **closed vocabulary** — the agent cannot invent new operators. It must declare everything that LLMs usually get wrong: coordinate system (`EPSG:4326`), axis order (`lat,lon`), units (`metre`), window size and slide, watermark, lateness, and a provenance id saying which reasoning episode created each operator. The dashboard panel "Active GS-IR plan" prints it in readable form. Version 1 is the human baseline; each promotion increments the version.

7.2 The planner (the "AI")

By default the planner is **rule-based**: a deterministic risk score from the evidence (slow fraction, mean speed, z-score, and prior memory facts) → label `congestion_incident` (risk ≥ 0.6) or `dense_traffic`. Optionally, a **free local LLM via Ollama** (e.g. llama3.2) fills the same typed slots in JSON; if it answers anything outside the schema, the system falls back to rules. A 2 s artificial delay simulates model latency. If it is an incident, the agent proposes a geofence: new circle around the cluster, or a TTL refresh if one already covers it.

7.3 The Geo-Verifier: catching "geo-hallucinations" before they run

To demonstrate this, the agent is made to deliberately corrupt 20% of its patches (adjustable live). The verifier (`verifier.py`) must catch them all. It checks 11 fault classes:

Injected fault	Real-world meaning	Check that catches it
swapped_axis_order	lat and lon swapped	centre outside the city, but inside if swapped
distance_in_degrees	radius 0.008 "degrees" instead of 900 m	units must be metre; radius below GPS accuracy
missing_crs / missing_datum_transform	no or foreign coordinate system	CRS mandatory, must be EPSG:4326
index_radius_incompatible	huge circle on tiny tiles	ring size k > 12 is too expensive
slide_greater_than_size	window slides 120 s, size 60 s	slide ≤ size, size multiple of slide
watermark_below_out_of_orderness	would drop most real late data	watermark + lateness vs observed disorder
de9im_unsatisfiable	"disjoint AND within" — impossible	known contradictory predicate pairs
precision_exceeds_accuracy	3 m geofence with 15 m GPS	radius ≥ positional accuracy
latency_budget_exceeded	plan cannot meet its p99 budget	estimated cost vs declared budget
invented_operator	agent makes up `near_ish()`	closed vocabulary

If rejected, the errors are fed back and the agent retries (up to 3 times). The dashboard button "Run fault-injection benchmark" runs 220 corrupted + 100 valid plans: **100% caught, 0% false rejections** (paper target H5: ≥ 95% and < 2%).

7.4 Governor, shadow run, promotion and hot-swap

- **Governor:** small geofences are auto-promoted; geofences with radius > 1.3 km (large area affected) are **held for human approval** on the dashboard. Humans approve *policies*, not every event — so humans are never in the per-event latency path.
- **Shadow:** the candidate plan is published as `stage=shadow`. Fastplanes evaluate it on the live stream in parallel, but its alerts are only counted ("would-be alerts"), never sent. After 20 s, if the alert rate and latency are sane, it is

promoted.

- **Hot-swap:** `stage=active` is published; every fastplane switches at the next window boundary, so no window is computed half with the old plan and half with the new one.
- **TTL:** temporary geofences expire after 6 minutes unless the incident is still there (then they are refreshed).

7.5 Geo-Episodic Memory (GEM)

After each decision, a **fact** is written to the `gem_facts` table: *where* (tiles), *when* (valid from/to), a claim, a confidence, and a **half-life** (5 min for incidents, 2 min for "clear"). When the agent reasons about a new hotspot, memory is filtered **spatially first** (tiles within 2 rings) and **temporally** (last hour), then confidence decays as `c₀ · 2^(−age/half-life)`; facts below 0.2 are hidden. Old knowledge fades instead of misleading the agent. Conflicting facts ("incident" and "clear" for the same place) are flagged.

7.6 Spatio-semantic cache + variogram ("Reason once, execute many")

Nearby places at the same moment usually deserve the same judgement (Tobler's law: near things are more related). The cache reuses a past decision if: (1) within **k* grid steps**, (2) within 120 s, (3) features very similar (cosine ≥ 0.92), (4) same regime (time of day + severity band). The clever part is **k***: it is learned from the agent's own decisions using a **semivariogram** — the average squared difference of decisions as a function of distance h. Where decisions are smooth over distance, k* grows; where they change quickly, k* = 0 (same tile only). The dashboard bar chart shows $\hat{\gamma}(h)$ and the current k*.

7.7 Distiller: turning repeated judgement into code

Once the agent has made enough labelled decisions (≥ 8 of each kind), a tiny **logistic-regression** model is fitted on four named features (slow fraction, normalised mean speed, log count, vehicles). If its F1 ≥ 0.8, it is proposed as a `distilled_filter` operator — verified and shadowed like any patch — and then runs inside the fastplane in microseconds (`DISTILLED_CONGESTION` alerts). If its agreement with later agent decisions drops below 70%, it is automatically **demoted** (`OPERATOR_DEMOTED`). Knowledge flows both ways.

8 · Storage and fault tolerance

8.1 TimescaleDB

TimescaleDB is PostgreSQL with time-series superpowers. Its **hypertables** automatically split a big table into time chunks, so "last 10 minutes" queries stay fast and old data is dropped by a **retention policy** (metrics 3 days, aggregates 14 days). It is normal SQL, so any PostgreSQL client works.

Table	Contents
`tile_aggregates` **hypertable**	One row per tile per window. Primary key (tile, window_end).
`alerts` **hypertable**	Every alert, with type, severity, location, plan version, provenance.
`agent_decisions` **hypertable**	Agent judgements and cache reuses.
`surprise_events` **hypertable**	Evidence bundles sent to the agent.
`ops_metrics` **hypertable**	Telemetry time series.
`gem_facts`	Geo-episodic memory.
`plan_versions`, `verifier_reports`	Audit trail: every plan, its diff, verification result and who/what created it.
`checkpoints`	Fastplane window state + Kafka offsets (fault tolerance).
`scenario_truth`	Injected incidents (for detection time).
`city_minute` (view)	City-wide per-minute summary: active tiles, observations, average speed.

8.2 Fault tolerance — eight mechanisms

Mechanism	What it protects against
1. Aligned checkpoints	Every 10 s (and before giving up a partition), each fastplane saves *window state + the Kafka offset it corresponds to* in one database transaction. On restart or rebalance, the new owner restores the state and seeks Kafka to that exact offset. Nothing is lost; nothing is counted twice in the state.
2. Kafka retention / replay	Events after the last checkpoint are still in Kafka and are simply re-read.
3. Idempotent producers	Network retries never create duplicate messages.
4. Commit after durable write	The sink commits Kafka offsets only after the database transaction commits.
5. Idempotent sink	Every result has a deterministic key and is written with `ON CONFLICT`. If a window is re-emitted after recovery, the row is overwritten, not duplicated.
6. Rewind-and-retry	If the database is down, the sink seeks back to the batch start and retries every 5 s.
7. Dead-letter queue	Poison messages are quarantined in `gps.dlq` instead of crashing consumers.
8. Restart policies & graceful shutdown	Docker restarts crashed services; on Ctrl+C / docker stop, services write a final checkpoint.

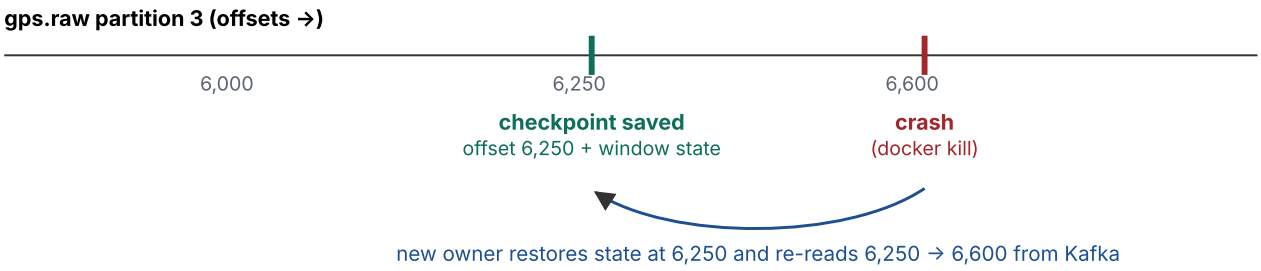


Figure 6 — Recovery. Re-emitted window results carry the same keys, so the sink overwrites instead of duplicating.

> **SAY THIS**
>
> "Delivery is at-least-once, but every write is idempotent, so the *effect* is exactly-once: a crash can cause some events to be reprocessed, but they never change the stored results twice."

8.3 Degradation ladder

If the fastplane falls behind (median latency > 2 s → level 1, > 10 s → level 2), it sheds optional work in a fixed order: first the exact geometry refinement, then shadow evaluation. The level is stamped on every result and shown on the dashboard ("degradation L0" = normal).

9 · Docker: how it all runs on Windows

Four words you need

- **Image** — a frozen package of an operating system + software (like a .iso or an installer). E.g. `apache/kafka:3.8.1`.
- **Container** — a running copy of an image, isolated like a tiny virtual machine but much lighter.
- **Docker Compose** — a file (`docker-compose.yml`) describing many containers, their settings, ports, and start order; one command starts them all.
- **Volume** — persistent storage that survives container restarts (our database lives in volume `pgdata`).

On Windows, Docker Desktop runs a small Linux virtual machine via WSL 2; all containers run inside it. That is why the project works the same on any computer: nobody has to install Kafka, Java, PostgreSQL or Python manually.

The containers in docker-compose.yml

Service	Image	Role	Port on your PC
`kafka`	apache/kafka:3.8.1	The message broker (KRaft mode).	9092
`timescaledb`	timescale/timescaledb:2.17.2-pg16	Database. Runs `db/init.sql` on first start.	5433
`kafka-ui`	provectuslabs/kafka-ui	Web UI to browse topics/messages.	8085
`init`	tessera-app (ours)	One-shot: builds the app image, creates 11 topics, schema, publishes plan v1, then **exits with code 0** (expected).	—
`producer`	tessera-app	Data source.	—
`fastplane` ×2	tessera-app	Stream processing (replicas = 2).	—
`gate`	tessera-app	Surprise gate + cache.	—
`slowplane`	tessera-app	Agent, verifier, governor.	—
`sink`	tessera-app	Kafka → TimescaleDB.	—
`dashboard`	tessera-app	Web dashboard.	8000

Start order is enforced with health checks: Kafka and the database must be healthy → `init` must finish successfully → then all pipeline services start. All our Python services share **one image** (`docker/Dockerfile`, Python 3.12) and differ only in the command they run.

Two network addresses for Kafka: inside Docker, services reach it at `kafka:29092`; from your Windows PC (e.g. a tool you run) it is `localhost:9092`. The broker advertises both listeners.

> **THE BUG YOU HIT ON FIRST RUN (GOOD STORY FOR "CHALLENGES FACED")**
>
> Originally seven services each had a `build:` section with the same image name. Newer Docker Desktop builds them in parallel, and they collided: *"image tessera-app:latest already exists"*. Fix: only `init` builds the image; every other service uses it with `pull_policy: never`, and `start.bat` runs `docker compose build init` before `docker compose up -d`.

10 · Every command and what its output means

Run these from the project folder (the one containing `start.bat`). In the VS Code / PowerShell terminal prefix with `.\`; in File Explorer just double-click.

start.bat

```
[1/3] Building the TESSERA image ...          ← docker compose build init (pip installs requirements)
      Starting containers...                  ← docker compose up -d
 ✔ Container tessera-kafka-1        Healthy    ← health check passed
 ✔ Container tessera-init-1         Exited     ← correct! one-shot setup finished with code 0
 ✔ Container tessera-fastplane-1    Started
[2/3] Waiting for the dashboard on http://localhost:8000 ...
[3/3] Pipeline is up.
```

status.bat

First a list of containers (all `running`, `init` `exited (0)`), then one table per consumer group. How to read the table:

```
GROUP            TOPIC    PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CONSUMER-ID              HOST
tessera-fastplane gps.raw  0          27985           27988           3    24323769b041-tessera-... /172.18.0.11
```

Column	Meaning
PARTITION	Which of the 6 partitions this row is about.
CURRENT-OFFSET	The last offset the group committed (“read and processed up to here”).
LOG-END-OFFSET	Newest message in the partition.
LAG	Difference = messages not yet processed. Small numbers (0–200) that go up and down are healthy; a number that grows forever means the consumer cannot keep up.
CONSUMER-ID	Which member owns this partition. The first 12 characters are the **container id**.
HOST	The container’s IP on Docker’s internal network.

Note: fastplane commits offsets every 10 s with its checkpoint, so its LAG can briefly show a few hundred — that is the gap since the last checkpoint, not a backlog. Partitions have different totals because busy parts of the city send more events (spatial keying).

scale.bat 3 — demonstrating consumer groups

Starts a third fastplane container in the same group. Kafka rebalances. Your real output:

```
PARTITION  ... LAG  CONSUMER-ID
2              155  2d1382b9b030-tessera-fastplane-12f6...   /172.18.0.12
3               68  2d1382b9b030-tessera-fastplane-12f6...   /172.18.0.12
4                1  aec77766f52b-tessera-fastplane-2195...   /172.18.0.7
5                3  aec77766f52b-tessera-fastplane-2195...   /172.18.0.7
0                3  24323769b041-tessera-fastplane-f52b...   /172.18.0.11
1                0  24323769b041-tessera-fastplane-f52b...   /172.18.0.11
```

Read it as: three different consumer ids, one group, two partitions each (2/2/2) — before it was 3/3. Partitions 2 and 3 moved to the new member and show a little lag while it restores their window state from the checkpoint and catches up; a few seconds later all lags are near 0. `scale.bat 2` returns to two consumers (Kafka rebalances again, 3/3).

demo-fault-tolerance.bat

```
--- before ---                     2 members, 3 partitions each
--- docker kill tessera-fastplane-1 --- instant crash: no clean shutdown, no final checkpoint
waiting 25 s ...                   Kafka waits for the session timeout (15 s) then rebalances
--- after the crash ---            ONE member now owns all 6 partitions
--- survivor log ---
[fastplane] INFO partitions [3, 4, 5] revoked - checkpointing before hand-off
[fastplane] INFO RECOVERED partition 0 from checkpoint: offset=12355, 6 open windows, watermark=...
[fastplane] INFO RECOVERED partition 1 from checkpoint: offset=2411, 6 open windows, ...
--- restarting the crashed instance ---  after rebalance: back to 3 + 3
```

Read it as: during a rebalance, a consumer first *checkpoints* the partitions it is giving away ("revoked"), then *restores* the ones it receives ("RECOVERED … offset=12355, 6 open windows"). For the crashed instance's partitions, the survivor resumes from the last checkpoint (≤ 10 s old) and re-reads the rest from Kafka. The dashboard keeps updating throughout; at most a short dip in the events/s chart.

demo-kafka.bat

1. **Topic list** — the 11 topics (plus Kafka's internal `__consumer_offsets`).
2. **Describe gps.raw** — `PartitionCount: 6`, and for each partition its `Leader` (broker 1), `Replicas` and `Isr` (in-sync replicas; 1 because we run one broker).
3. **Consumer group** — same table as status.bat.
4. **Raw events** — lines like `Partition:4 8831aa…ffff {"vid":"taxi-0123","ts":…,"speed":34.2,"lat":39.91,"lon":116.40,"crs":"EPSG:4326","axis":"lat,lon"}`. The part before the JSON is the **key** (an H3 cell id) — show that events with the same key are always in the same partition.
5. **Aggregates** — `{"window_start":…,"window_end":…,"tile":"8831…","n_obs":42,"vehicles":9,"mean_speed":31.5,"slow_frac":0.02,…}`.
6. **Alerts** — JSON alerts with `type`, `message`, `plan_version`, `provenance_id`.

logs.bat [service] — decoding the log lines

Log line	Meaning
`[producer] sent 200 ev/s, active incidents=1, delayed=60, ooo=1226 very_late=102 malformed=49`	Send rate; incidents currently active; events held back to arrive late; running counts of out-of-order, very late and deliberately corrupted events.
`[producer] INJECTED incident inc_63fd2b8a at (39.935, 116.368) r=711m for 240s`	Ground truth: where the hidden incident is.
`[fastplane] 110 ev/s | p50 731 ms p99 744 ms | partitions [0, 1, 2] | late 0 dlq 22 | alerts 0 | plan v2 | shadow v3=7`	This instance's rate; latency from producer send to processing; which partitions it owns; late & dead-letter counts; geofence alerts; active plan; would-be alerts of the plan in shadow.
`[fastplane] HOT-SWAP plan v1 -> v2 (1 geofences, 0 distilled operators)`	New plan loaded at a window boundary.
`[gate] windows=12 tests=4394 escalated=11 invocations=2 cache_hits=2 k*=4 deferred=0`	Windows evaluated; tile tests run; tiles flagged by BH; agent calls; cache reuses; current reuse radius; bundles waiting for a token.
`[gate] ESCALATE ev_2e70…: 2 tiles z=14.2 -> slow plane`	An evidence bundle was sent to the agent.
`[slowplane] episode ep_a856…: congestion_incident risk=1.00 (slow fraction 74%, …) [2.0s]`	One reasoning episode, its decision, rationale, and time taken.
`[slowplane] VERIFIER rejected candidate (swapped_axis_order): …`	A corrupted patch was blocked; the agent repairs and retries.
`[slowplane] SHADOW v2 started … / PROMOTED v2 after 20s shadow (5 would-be alerts)`	Safe rollout of a new plan.
`[slowplane] GOVERNOR: holding v3 for approval`	Large geofence waits for a human click.
`[slowplane] DISTILLER: 62 episodes, F1=0.86 -> proposing dm_f7f0…`	Agent judgement compiled into a fast operator.
`[sink] stored 70 records/s`	Database write rate.

Press `Ctrl`+`C` to stop following logs (the services keep running).

Other commands

Command	What it does / expected output
`run-tests.bat`	Runs 11 unit tests inside Docker: `11 passed`.
`stop.bat`	Stops all containers; Kafka data and database are kept. `start.bat` resumes.
`reset.bat`	Type YES: deletes containers, Kafka topics and the database volume — fresh start.

`docker compose ps`	List of TESSERA containers and their state.
`docker restart tessera-timescaledb-1`	Database outage test: sink logs “write failed … rewinding batch and retrying”, then continues; no data lost.
`docker restart tessera-kafka-1`	Broker outage test: services log “waiting for Kafka” then reconnect on their own.
`docker kill tessera-slowplane-1`	Agent dies → the fast plane keeps processing and alerting at full speed (P1). Restart: `docker start tessera-slowplane-1`.

11 · Dashboard tour, panel by panel



Figure 7 — The live dashboard at http://localhost:8000.

Panel	What it shows / how to read it
Header chips	Kafka connection (green dot), active plan version, plan in shadow (if any), planner type, event time of the last closed window.
INGESTED	Events/s processed by the fast plane (15 s average); produced rate; number of consumers in the group.
FAST-PLANE P99	99th-percentile time from producer send to processing (includes Kafka transfer and batching, ~0.3–0.6 s here); p50; degradation level.
AGENT INVOCATIONS	Agent calls vs tile tests vs escalated tiles — the cost-control story in one box.
CACHE REUSE	Decisions reused without a model call; k*; hit rate.
LATE / DLQ	Events sent to side output / dead-letter queue.
VERIFIER BLOCKS	Patches rejected; total patches; promoted; held for approval.
Map	Hexagons = tiles with data in the last window. *Slow fraction*: green → red. *Density*: observations. *Kafka partition*: colour = partition (shows spatial keying). *Consumer*: colour = which fastplane container processes it. Red circle = active geofence, dashed amber = shadow, white dashed = injected incident (truth), purple dots = agent decisions.
Live alerts	Filterable feed: GEOFENCE_ENTER (fast plane), HOTSPOT_DETECTED (gate), AGENT_DECISION (agent), CACHED_DECISION (cache), DISTILLED_CONGESTION (distilled operator), VERIFIER_REJECTED, PLAN_PROMOTED, APPROVAL_REQUIRED.

Throughput & latency	Green: events/s. Yellow: fast-plane p99. Purple dashed: agent latency ×100. The key visual for H1.
Demo controls	Inject 30 s agent latency / restore; geo-hallucination rate 0/20/60%; pending approvals with Approve/Reject.
Kafka topics & partitions	Every topic, each partition's end offset, total messages.
Consumer groups	State (STABLE), members → assigned partitions, total lag.
Active GS-IR plan	The current plan in readable form.
Plan history	Every version, stage (shadow/active/held/rejected), disposition and reason.
Geo-Verifier catch log	Which fault classes were caught live; benchmark button.
Decision-field variogram	γ(h) per grid distance; green bars = within reuse radius k*.
Recent agent decisions	Label, source (agent/cache), risk, rationale.
Incident detection	For each injected incident, seconds until the first matching detection. Typical: 8–55 s (windows are 60 s long, so detection needs part of a window to fill).
Fault tolerance	Checkpoint per partition: offset, owning instance, age (should stay ≤ ~10 s).
Storage	Row counts per table; latest memory facts with decayed confidence.

12 · A 7-minute live demo script

Start the pipeline 5 minutes before you present so incidents, plans and history are already on screen.

1. **Problem (45 s).** “Spatial streams like taxi GPS need real-time answers. Stream engines are fast but blind; AI agents are smart but slow. Putting an LLM per event fails on arithmetic. TESSERA separates the two speeds.”
2. **Architecture (60 s).** Show Figure 2. “Producer → Kafka → fast plane → gate → slow plane, everything through 11 Kafka topics, stored in TimescaleDB, shown here.”
3. **Dashboard (60 s).** Point at the map: incident (white circle) → red tiles → hotspot alert → agent decision → promoted geofence → GEOFENCE_ENTER alerts. Click *Kafka partition* then *Consumer*.
4. **Kafka (60 s).** Scroll to topics and consumer groups. Run `.\scale.bat 3`, show the 2/2/2 table, explain rebalance. `.\scale.bat 2`.
5. **Fault tolerance (75 s).** Run `.\demo-fault-tolerance.bat`. Show “RECOVERED partition … from checkpoint”. Say the exactly-once-effect sentence (chapter 8).
6. **Decoupling (45 s).** Click *Inject 30 s agent latency*. Point at the chart: the agent line jumps, p99 stays flat. Restore.
7. **Safety (45 s).** Click *Run fault-injection benchmark*: 100% caught, 0% false rejection. Show the plan history (verify → shadow → promote).
8. **Close (30 s).** Requirements table (chapter 15) and one honest limitation (chapter 16).

BEFORE THE PRESENTATION

- Close other programs using ports 8000, 8085, 9092, 5433 (e.g. your old jarvis-backend container on 8000).
- The map tiles and chart library load from the internet. Without internet the data layers still work but the background map is blank.
- If something looks frozen: `.\status.bat`; if needed `.\stop.bat` then `.\start.bat`.

13 · Viva questions and answers

Q1. Why Kafka and not just a database or a REST API between services?

Kafka decouples producers from consumers in time and speed: the producer does not wait for anyone, consumers read at their own pace, and messages are retained so a crashed consumer can replay. It gives ordering per partition, horizontal scaling through partitions and consumer groups, and multiple independent readers of the same data (sink, gate and dashboard all read `tile.aggregates`). A REST chain would couple every service's availability and speed to the next one.

Q2. How did you choose the partition key?

The coarse H3 cell (resolution 6, ~3 km). Same area → same partition → same consumer, so per-tile window state is never split. A random key would balance load perfectly but break per-tile aggregation; a very fine key would scatter neighbours. The trade-off is some imbalance, visible in different partition sizes.

Q3. What happens if you run 8 fastplane consumers?

Only 6 get partitions; 2 sit idle as hot standbys, ready to take over in a rebalance. Parallelism is capped by the partition count.

Q4. What is the difference between event time and processing time? Why does it matter?

Event time is when the GPS reading was taken; processing time is when we handle it. Using event time makes results independent of network delays and replays — a recovered consumer recomputes the same windows.

Q5. Explain the watermark.

A watermark W is the claim "no more events older than W should arrive". We use max event time − 4 s. A window [s, e) is emitted when e + 3 s lateness ≤ W. Larger values = more complete but later results; smaller = faster but more late events. Late ones are kept in `gps.late`.

Q6. Why sliding windows (60 s / 10 s) instead of tumbling windows?

Tumbling 60 s windows would update the picture only once a minute, and an incident starting mid-window would be diluted. Sliding windows give a fresh "last minute" every 10 s, so detection is faster and smoother.

Q7. Which aggregation algorithms did you implement?

Per-tile count, distinct vehicles, mean and standard deviation via running moment sums, slow fraction; then Getis-Ord G^*_i local spatial statistic, Benjamini–Hochberg FDR control, connected-component clustering, EWMA baselines, an empirical semivariogram, and logistic-regression distillation.

Q8. Why G* and not just "slow fraction > 50%"?

A fixed threshold ignores the city context (rush hour) and fires on single noisy tiles. G* measures whether a tile *and its neighbourhood* are unusually high relative to the whole city, giving a calibrated z-score and p-value.

Q9. Why is FDR control needed?

We test hundreds of tiles every 10 s. Without correction, false alarms scale with the number of tiles and would wake the agent constantly. BH bounds the expected share of false discoveries at q = 1%.

Q10. How do you guarantee no data loss when a consumer crashes?

Window state and the Kafka offset are checkpointed together every 10 s. The next owner restores that state and re-reads from that offset; Kafka still holds the newer events. Output keys are deterministic and the sink upserts, so replays do not duplicate rows.

Q11. Is it exactly-once?

Delivery is at-least-once (some events can be re-read after a crash); idempotent writes make the stored effect exactly-once. True end-to-end exactly-once would need Kafka transactions across consume–process–produce, which is listed as future work.

Q12. What if the database goes down?

The sink's transaction fails, it does not commit the Kafka offset, seeks back to the batch start and retries every 5 s. Kafka buffers everything meanwhile.

Q13. Where is the AI, and is it really an LLM?

In the slow plane. By default it is a deterministic rule-based reasoner so the demo is free and reproducible; it can be switched to a free local LLM via Ollama by setting `OLLAMA_URL`. The architecture is identical: the model only fills typed slots and its output must pass the verifier. The point of the project is the architecture that makes any model safe and non-blocking.

Q14. How do you prove the agent does not slow the stream?

Live: inject 30 s agent latency and the fast-plane p99 stays flat. Structurally: the fast plane never calls the agent; they only exchange Kafka messages, and plan changes are applied asynchronously at window boundaries.

Q15. What is GS-IR and why not let the agent write Python code?

A typed, closed-vocabulary JSON plan with mandatory CRS, axis order, units and time semantics. Free code cannot be statically verified for geographic correctness; a typed plan can. It also makes every change diffable, versioned and auditable.

Q16. What is a shadow run?

The candidate plan runs on the live stream alongside the active one, but its alerts are only counted. If behaviour is sane for 20 s it is promoted. It catches problems static checks cannot, such as a geofence that would fire thousands of alerts.

Q17. What is the variogram doing in a cache?

It measures how quickly the agent's decisions change with distance. If decisions 2 tiles apart are nearly identical, a cached decision may be reused up to 2 tiles away ($k^* = 2$). If they differ, reuse is limited to the same tile. The reuse radius is learned from data, not hand-picked.

Q18. Where are humans in the loop?

At the policy level: large-area changes (geofence > 1.3 km) wait for Approve/Reject on the dashboard. Humans never approve individual events, so they are not a latency dependency.

Q19. How do you measure correctness of detection?

The producer publishes ground truth to `scenario.truth`. The dashboard computes, for each injected incident, the time until the first hotspot/decision near it. Typical values 8–55 s with 60 s windows.

Q20. Why TimescaleDB instead of plain PostgreSQL or MongoDB?

Our data is time-stamped measurements queried by recent time ranges. Hypertables keep those queries fast as data grows and support retention policies, while staying standard SQL with joins (e.g. alerts vs ground truth).

Q21. Why Python and not Flink?

To run on a student laptop under Windows with modest memory. The concepts Flink provides — event time, watermarks, keyed state, aligned checkpoints, rebalancing — are implemented explicitly so they can be shown and explained. Moving the fast plane to Flink/PyFlink is the scale-up path.

Q22. What was the hardest problem?

Good answers: (a) the ring-size formula giving false negatives on distorted H3 cells — fixed with a measured bound and a property test; (b) making plan updates consistent across instances — solved with a compacted broadcast topic and watermark-aligned hot-swap; (c) the Docker parallel-build collision on Windows.

14 · Code map: what every file does

File	Responsibility
Windows scripts & infrastructure	
`start.bat / stop.bat / reset.bat`	Build & start everything / stop / wipe.
`status.bat / logs.bat`	Health and consumer groups / follow logs.
`scale.bat / demo-kafka.bat / demo-fault-tolerance.bat`	Kafka and recovery demonstrations.
`run-tests.bat`	Unit tests in Docker.
`docker-compose.yml`	All containers, ports, env variables, start order, replicas.
`docker/Dockerfile`	The single Python 3.12 image for all our services.
`requirements.txt`	confluent-kafka, h3, numpy, psycopg2, fastapi, uvicorn, pydantic, pytest.
`db/init.sql`	Tables, hypertables, retention, indexes, the `city_minute` view.
Core library — `tessera/`	
`config.py`	Every topic name, partition count, group id and tunable parameter (all overridable by environment variables).
`kafka_utils.py`	Creates topics; builds idempotent producers and manual-commit consumers; JSON encoding; metrics publisher; graceful-shutdown handler.
`db.py`	Database connection with automatic reconnect and retry.
`geo.py`	H3 tiling, haversine distance, cell geometry, the no-false-negative ring bound, event validation/normalisation.
`windows.py`	Event-time sliding windows, watermarks, lateness, idle advancement, snapshot/restore for checkpoints, per-tile aggregation.
`stats.py`	G*, p-values, Benjamini–Hochberg, clustering, EWMA, decision variogram, token bucket, logistic distiller.
`gsir.py`	The plan format: vocabulary, baseline plan v1, applying patches, rendering as text.
`verifier.py`	Static plan verification and the 11-class fault-injection suite.
`memory.py`	Spatio-semantic cache and Geo-Episodic Memory with decay.
Services — `tessera/services/` (each runs in its own container)	
`topic_init.py`	The `init` container: topics, schema, plan v1.
`producer.py`	Data source: taxi simulation, incidents, out-of-order/late/malformed events, spatial keys; optional T-Drive replay.
`fast_plane.py`	Stream processing: consume, normalise, tile, window, aggregate, geofences, distilled operators, plan hot-swap, shadow evaluation, checkpoints, rebalance handling, degradation ladder.
`gate.py`	Surprise gate: G* + BH, clustering, cache lookup, token bucket, escalation.
`slow_plane.py`	Agent: planner (rules/Ollama), memory, patch building, verifier + repair, governor, shadow → promote, distiller and demotion, human controls.
`sink.py`	Kafka → TimescaleDB with idempotent upserts and rewind-on-failure.
Dashboard and tests	
`dashboard/app.py`	FastAPI server: live Kafka consumer, Kafka admin (partitions, groups, lag), history queries, control endpoint, verifier benchmark.
`dashboard/static/index.html`	The web page: Leaflet map, Chart.js charts, alert feed, tables, buttons.
`tests/test_core.py`	11 tests: ring bound (P3), normalisation, windows, late events + checkpoint round-trip, G*/BH, variogram, distiller, plan verification, fault suite.

15 · Requirements checklist

Requirement	How TESSERA meets it	Show it with
Complete pipeline: Data Source → Kafka → Stream Processing → Storage → Analytics/Dashboard	producer → Kafka → fastplane/gate/slowplane → sink → TimescaleDB → dashboard	Figure 2, dashboard
Kafka as the central platform	11 topics; every service-to-service hop is a topic	Topics table, Kafka UI
≥ 1 producer and consumer	5 producers, 5 consumer services	Code map
≥ 2 topics	11 topics	`demo-kafka.bat`
Partitioning	6 partitions on `gps.raw` and `tile.aggregates`, keyed by H3 cell	Map “Kafka partition”
Consumer groups	4 groups; fastplane ×2, scalable to 6	`scale.bat 3`
Window-based operation	60 s sliding windows, 10 s slide, watermark, lateness, side output	Map, late counter
Aggregation / summarisation algorithm	Per-tile statistics; G*; BH-FDR; clustering; variogram; distiller	Hotspot alerts
Fault tolerance / recovery	Checkpoints + offsets, replay, idempotent producers and sink, DLQ, retries, restart policy	`demo-fault-tolerance.bat`
Database / time-series store	TimescaleDB hypertables with retention	Storage panel, SQL client
Real-time visualisation or alerts	Live map, charts, 8 alert types	Dashboard

Paper hypotheses vs what you can show

ID	Hypothesis	In this prototype
H1 / P1	Data-path p99 unchanged under 30 s agent delay	Live button; chart stays flat
H3 / P2	Agent calls far fewer than events	“tile tests” vs “agent invocations” KPI (thousands vs single digits)
H4	Variogram-calibrated reuse	Cache hits, k*, variogram panel
H5 / P5	≥ 95% faults caught, < 2% false rejections	Benchmark: 100% / 0%
P3	No false negatives from spatial pruning	Unit test with corrected bound

16 · Honest limitations & future work

Examiners trust a project more when you state its limits yourself.

- **Python fast plane, not Flink.** Fine for ~200 events/s on a laptop; the paper targets millions/s on a cluster. Future: port to Flink/PyFlink with RocksDB state.
- **Single Kafka broker** (replication factor 1). Production: 3 brokers, replication 3, `min.insync.replicas=2`.
- **Rule-based planner by default**; LLM optional via Ollama. Agent "hallucinations" are simulated by fault injection to test the verifier.
- **At-least-once + idempotent writes** rather than Kafka transactions.
- **Synthetic data** by default; Microsoft T-Drive replay is supported (put files in `data/tdrive`).
- **Shadow promotion** checks alert rate and latency, not equivalence with an offline oracle.
- **Fastplane p99 ≈ 0.3–0.6 s** end-to-end includes Kafka batching and the consumer poll interval; it is stable, which is what H1 is about, but could be lowered by tuning `linger.ms` and poll timeouts.

17 · Glossary

Term	Meaning
Spatial stream	Never-ending sequence of events that each have a location and a time.
Fast plane / slow plane	Deterministic per-event processing / agent reasoning about the plan.
H3, tile, resolution	Uber's hexagonal global grid; one hexagon; size level (8 ≈ 0.5 km edge, 6 ≈ 3 km).
k-ring	A tile plus all tiles within k steps.
CRS / EPSG:4326	Coordinate reference system; 4326 = ordinary GPS latitude/longitude (WGS 84).
Axis order	Whether coordinates are written lat,lon or lon,lat — a classic source of bugs.
Haversine	Formula for distance on a sphere (in metres) between two lat/lon points.
Window / slide	Time bucket length / how often a new bucket starts.
Watermark	Event-time point up to which the stream is considered complete.
Allowed lateness	Extra grace period after the watermark before a window is finalised.
Side output / DLQ	Separate topic for late events / for malformed records.
Offset, lag	Position in a partition / unprocessed messages.
Rebalance	Kafka reassigning partitions when group membership changes.
Idempotent	Doing it twice has the same effect as doing it once.
Checkpoint	Saved snapshot of state + input position for recovery.
Getis-Ord G*	Local hotspot statistic comparing a neighbourhood sum to the global expectation; gives a z-score.
p-value, FDR, BH	Probability by chance; expected share of false alarms among alarms; procedure that controls it.
EWMA	Exponentially weighted moving average (slowly adapting baseline).
Semivariogram γ(h)	Half the mean squared difference of values at distance h; measures spatial smoothness.
Token bucket	Rate limiter: tokens refill at a fixed rate; each call spends one.
GS-IR	Geo-Stream Intermediate Representation — TESSERA's typed plan format.
Shadow run / promotion / hot-swap	Test a plan on live data without effect / make it active / switch running instances to it without restart.
GEM	Geo-Episodic Memory: georeferenced, time-bounded, decaying facts.
Distillation	Compiling repeated agent judgements into a cheap model (logistic regression) that runs inline.
Hypertable	TimescaleDB table automatically partitioned by time.
KRaft	Kafka's built-in consensus mode that replaces ZooKeeper.
Container, image, compose, volume	Running isolated program / its frozen package / multi-container definition / persistent storage.